Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: PROJECT EVALUATION (24CS2235F) A.Y. 2026 - 2027

TITLE OF THE PROJECT	AI-Driven Personalized Healthcare Decision Support and Remote Patient Monitoring Platform	
S. No.	**University ID**	**Name**
1	2420030036	IKSHWAK
2	2420030261	MUKESH
3	2420030407	HARSHA
4	2420030409	MOHIT
Name of the Guide		B Dwarakanath

Abstract

Abstract

The Mini Programming Language Compilation Framework is a lightweight compiler-based system developed to demonstrate the complete process of converting high-level source code into machine-understandable instructions. The primary objective of the framework is to provide a simple and educational implementation of compiler design concepts without the complexity associated with industrial programming language compilers. The proposed framework supports a basic programming language containing fundamental constructs such as variables, arithmetic expressions, conditional statements, loops, and input/output operations.

The framework consists of several major phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation. During lexical analysis, the source program is divided into meaningful tokens such as keywords, identifiers, operators, constants, and delimiters. The syntax analyzer verifies whether these tokens follow the grammatical rules of the language and generates an appropriate parse structure. Semantic analysis then validates the meaning of the program by checking issues such as undeclared variables, incompatible data types, and invalid operations.

After successful analysis, the compiler generates an intermediate representation of the source program. This intermediate code provides an abstraction between the source language and the target machine, making the framework easier to extend to different target platforms. Basic optimization techniques can be applied to eliminate unnecessary operations and improve execution efficiency. Finally, the code generation phase converts the optimized intermediate representation into target instructions or executable-style code.

The framework also includes error detection and reporting mechanisms to help users identify and correct errors during compilation. By integrating all major compiler phases into a single system, the project provides a practical understanding of how modern compilers process programming languages. The proposed

framework can serve as an effective academic tool for students studying compiler design, programming languages, and system software. In the future, the framework can be enhanced by supporting advanced data types, functions, arrays, object-oriented features, sophisticated optimization algorithms, multiple target architectures, and an interactive compiler development environment.

Signature of the Guide HOD